

```
# 📄 Bug Report 2.0 & Feature Request Specification

**Target System:** MR.X.Change — Mobile Exchange & Inventory
**Target URL:**
[https://mrxchange.vercel.app/profit-expense-statistic] (https://mrxchan
ge.vercel.app/profit-expense-statistic)
**Date:** September 30, 2026
**Document Type:** Bug Diagnostic Report & Feature Specification
Document

---

## 🚨 Section 1: Bug Report - Duplicate Device Intake & Calculation
Inflation

### 1. Problem Statement & Observed Discrepancy
When adding a new device in "Add Mobile Entry" with a specified
Paid Amount (e.g., ₹3,600) and Paid By Super Admin (*Jeet
Khubchandani*), the system records duplicate entries under the
"Device Intake Investment History" table.

#### Audit Discrepancy Breakdown:
| Section | Recorded Entries | Amount Shown | Integrity Status |
| :--- | :--- | :--- | :--- |
| Capital Contribution Records | 1 Entry (`INV-602324`) | ₹3,600 |
✓ Correct |
| Device Intake Investment History | 2 Devices (Duplicate Apple
iPhone 13) | ₹3,600 + ₹3,600 = ₹7,200 | ✖ Corrupted (2x entries) |
| Total Invested Metric | Calculated from combined duplicates |
₹10,800 | ✖ Inflated by ₹7,200 |

---

### 2. In-Depth Root Cause Diagnosis

#### Cause A: SQL/ORM Query `JOIN` Duplication on Uploaded Images
(Primary Cause)
* The form mandates uploading at least 2 images (*1. Front Photo*,
*2. Back Photo*).
* In the database, the `devices` table is linked to `device_images`
(one-to-many relation).
```

* When querying ****Device Intake History****, the backend executes a ``LEFT JOIN`` on ``device_images`` without using ``DISTINCT`` or ``GROUP BY device_id``.

* ****Outcome:**** Because 2 images are attached to the device, the SQL query produces 2 rows for that single device.

Cause B: Missing Debounce / Double Submission on Form Button

* The ****"Add Mobile Device"**** button lacks a disabled loading state. If clicked twice or if both ``form.onsubmit`` and ``button.onclick`` fire, two identical POST requests hit the API simultaneously.

Cause C: Dual Database Hook Execution

* The backend controller inserts an investment ledger record, while an automated ORM hook (e.g. Prisma middleware or Mongoose ``post-save`` hook) also creates an entry automatically.

3. Recommended Resolution Strategy for Developers

Fix 1: Fix Backend SQL / ORM Query (Immediate Fix)

```
```sql
-- SQL Fix: Prevent duplication from child tables (images/attachments)
SELECT DISTINCT ON (d.id)
 d.id,
 d.created_at AS date,
 CONCAT(d.brand, ' ', d.model) AS device_name,
 d.paid_by_name,
 d.paid_amount
FROM devices d
LEFT JOIN device_images di ON di.device_id = d.id
WHERE d.paid_by_admin_id = :partnerId
ORDER BY d.id, d.created_at DESC;
```
```

Fix 2: Add Frontend Double-Click & Loading Guard

```
```jsx
const [isSubmitting, setIsSubmitting] = useState(false);

const handleAddMobile = async (e) => {
 e.preventDefault();
 if (isSubmitting) return; // Prevent double submission

```

```

 try {
 setIsSubmitting(true);
 await api.createMobileEntry(formData);
 closeModal();
 refetchPartnerHistory();
 } catch (err) {
 toast.error("Failed to add device: " + err.message);
 } finally {
 setIsSubmitting(false);
 }
 };
 ...

```

```


```

## ## 🌟 Section 2: Feature Request Specification - "Mobile Intake History" in Add Inventory Tab

### ### 1. Feature Overview & Objective

In the **"Add Inventory"** tab, introduce a dedicated **"📖 Mobile History"** action button. This button will open a comprehensive audit ledger showing complete historical logs of all mobile devices previously added as well as newly added devices in real time.

```

```

### ### 2. UI / UX Design Specifications

#### #### A. Placement & Trigger

\* **Location:** Top header of the **"Add Inventory"** page, placed next to the *"Add Mobile Device"* button.

\* **Button Label:** *"📖 Mobile Intake History" / "View Added Devices Log"*

\* **Interaction:** Clicking opens a modal or slide-over drawer with search, filter, and pagination capabilities.

#### #### B. Data Table Columns

The history view will display the following attributes for every mobile device:

- Device ID / Unique Code:** (e.g., *"DEV-98421"*, *"INV-602324"*)
- Device Brand & Model:** (e.g., *"Apple iPhone 13 (128GB - Midnight Black)"*)

3. **\*\*Date & Timestamp:\*\*** Date and time the device was recorded.
4. **\*\*Paid Amount (₹ INR):\*\*** Amount invested to purchase/intake the phone.
5. **\*\*Paid By (Super Admin):\*\*** Name of the investing partner (e.g., *\*Jeet Khubchandani\**).
6. **\*\*Added By (Staff/Admin):\*\*** The logged-in user who submitted the entry.
7. **\*\*Current Status:\*\*** Badge showing status (``In Stock``, ``Booked``, ``Under Repair``, ``Sold``).
8. **\*\*Attached Images:\*\*** Thumbnail preview of the uploaded photos.

#### #### C. Filter & Search Features

- \*  **\*\*Search Bar:\*\*** Real-time search by Model, IMEI, or Code.
- \*  **\*\*Date Range Filter:\*\*** From Date to To Date.
- \*  **\*\*Admin Filter:\*\*** Filter by investing partner.

---

### ### 3. Technical Implementation Blueprint for Developers

#### #### Step 1: API Endpoint (``GET /api/inventory/history``)

```
```javascript
// Backend Route: GET
/api/inventory/history?page=1&limit=20&search=iphone
router.get('/inventory/history', async (req, res) => {
  const { page = 1, limit = 20, search, adminId } = req.query;

  const query = {
    ...(search && {
      $or: [
        { model: { $regex: search, $options: 'i' } },
        { brand: { $regex: search, $options: 'i' } },
        { deviceCode: { $regex: search, $options: 'i' } }
      ]
    }),
    ...(adminId && { paidByAdminId: adminId })
  };

  const devices = await Device.find(query)
    .sort({ createdAt: -1 })
    .skip((page - 1) * limit)
    .limit(Number(limit))
    .populate('paidBy', 'name email');
```

```

    const total = await Device.countDocuments(query);
    res.json({ success: true, data: devices, total, page });
  });
  ...

```

Step 2: React Component Structure

```

```jsx
import React, { useState } from 'react';
import MobileHistoryModal from './MobileHistoryModal';

export default function AddInventoryPage() {
 const [isHistoryOpen, setIsHistoryOpen] = useState(false);

 return (
 <div className="inventory-header flex justify-between items-center p-4">
 <h1 className="text-xl font-bold">Add Inventory</h1>

 <div className="actions flex gap-3">
 { /* NEW FEATURE BUTTON */ }
 <button
 onClick={() => setIsHistoryOpen(true)}
 className="btn bg-gray-100 hover:bg-gray-200 text-gray-800 px-4 py-2 rounded-lg font-medium border"
 >
  Mobile History
 </button>

 { /* Existing Add Button */ }
 <button
 onClick={openAddModal}
 className="btn bg-blue-600 text-white px-4 py-2 rounded-lg font-medium"
 >
 + Add Mobile Entry
 </button>
 </div>

 { /* History Modal Component */ }
 {isHistoryOpen && (
 <MobileHistoryModal onClose={() => setIsHistoryOpen(false)} />
)}
 </div>
)
}

```

```

 </div>

);
}
...

Step 3: Automatic Real-Time Cache Update
When a user adds a mobile through `"Add Mobile Entry"`, automatically
trigger a query revalidation:
```javascript
// On successful submission of new mobile:
queryClient.invalidateQueries(['inventory-history']);
```

📋 Summary Checklist for Developers
- [] Fix SQL `DISTINCT` on image joins to eliminate duplicate intake
entries.
- [] Implement `isSubmitting` button disable state on the Add Mobile
form.
- [] Add the `📋 Mobile History` button in the Add Inventory tab
header.
- [] Build the History modal table with search, date filters, and
image previews.
- [] Connect automatic re-fetch so newly added phones appear
immediately in the history.

```